Tree-Based Models From Scratch: Benchmark Report

Student: Abdul Azeem | **Roll Number:** 24i-2013 | **Section:** BAI-5A | **Course:** Machine Learning

1. Overview

In this assignment, we built three tree-based machine learning algorithms completely from scratch using Python and NumPy: a Decision Tree, a Random Forest, and an AdaBoost classifier (SAMME multiclass). We tested all models on the Multiple Features dataset (mfeat-factors.csv) from UCI and OpenML. This dataset contains 2000 handwritten digit images from 0 to 9 (200 images for each digit) described by 216 numerical attributes. All parameters were calculated using the student roll number **24i-2013**: alpha = 1.3, base b = 3, lambda = 0.06, q = 0.13, stop_eps = 0.15, noise sigma = 0.04, and random seed = 2013. We compared the models on accuracy, bias, variance, noise robustness, feature importance, and verified them against scikit-learn.

Parameter	Formula	Value	Role in Our Implementation
alpha	1 + (3 / 10)	**1.3000**	Power exponent in the roll-based entropy formula H(p)
base b	2 + (1 mod 3)	**3**	Logarithm base used in entropy computation
lambda	0.01 * (2+0+1+3)	**0.0600**	Penalty weight subtracted from information gain based on feature correlation
quantile q	clip(13 / 100, 0.05, 0.95)	**0.1300**	Percentile split threshold candidate in quantile-based splitting
stop_eps	0.05 * max(1, 2013 mod 10)	**0.1500**	Minimum information gain threshold required to keep splitting
sigma	0.01 * max(1, 2013 mod 7)	**0.0400**	Standard deviation of Gaussian noise added to test robustness
seed	roll = 2013	**2013**	Base random seed for all random number generators

2. Data and Setup

Stratified Data Split: We divided the 2000 samples into three splits using only NumPy: 60% training (1200 samples, 120 per digit), 20% validation (400 samples, 40 per digit), and 20% test (400 samples, 40 per digit).
Feature Standardization: Features were standardized by subtracting the mean and dividing by the standard deviation. These statistics were calculated only on the training set. If we used statistics from the whole dataset, test set information would leak into the training process (data leakage). This would produce false, over-optimistic results that would fail on new data.
Exploratory Data Analysis: We computed a 2D PCA projection using Singular Value Decomposition (np.linalg.svd). The projection shows that digits 0 and 1 are easy to separate because they form clear isolated clusters. In contrast, digits 3, 5, and 8 overlap heavily in the center, making them difficult for a simple linear model to separate. A suite of 9 assert tests checked pure node entropy, probability sums, tree depth, and bootstrap size without any errors.

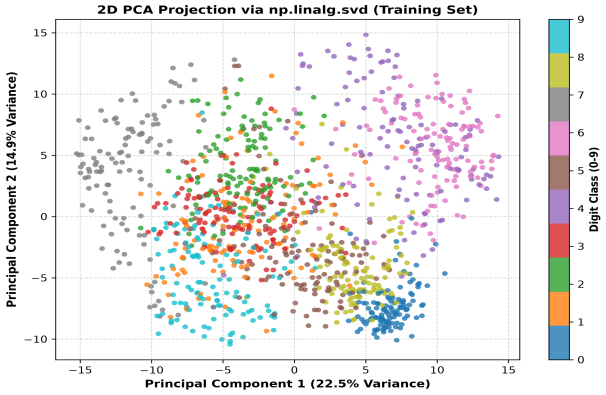


Figure 1: 2D PCA projection computed with np.linalg.svd.

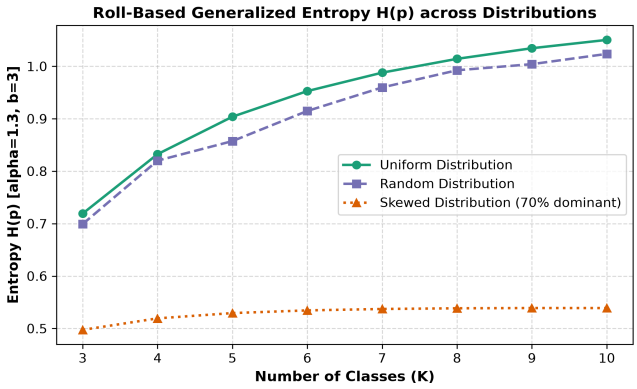


Figure 2: Roll-based entropy H(p) across distributions.

3. Methods and Design Choices

Decision Tree: We built the decision tree recursively using our custom roll-based entropy formula: H(p) = - sum(p_i^1.3 * log_3(p_i + 1e-12)). To choose splits, we subtract a penalty from information gain: IG_roll(f, t) = IG(f, t) - 0.06 * rho(f). Here rho(f) is the mean absolute Pearson correlation between feature f and the 10 one-hot class indicator vectors. We implemented both quantile splitting (threshold at the 13th percentile) and vectorized exhaustive splitting. Exhaustive splitting sorts each feature once and evaluates all thresholds quickly using cumulative class counts.

Random Forest: We created an ensemble of decision trees trained on bootstrap samples (drawn with replacement). At every split, the tree picks the best feature from a random subset of size sqrt(d) = sqrt(216) = 15 features. We compared majority voting against probability averaging. Out-of-bag (OOB) samples were tracked to calculate an accurate validation estimate.

AdaBoost (SAMME Multiclass): We implemented discrete multiclass boosting with weak decision trees of depth 1 to 3. In each round, sample weights are updated using alpha_m = ln((1 - error)/error) + ln(K - 1), where K = 10 classes. Hard-to-classify samples get higher weights so subsequent trees focus on them. An early stopping rule halts training when validation accuracy stops improving.

4. Experimental Results

Main Comparison Table (Table 1): We evaluated all four models over 5 different random seeds (2013 to 2017) on the exact same split. All metrics were calculated manually using custom functions.

Model	Test Accuracy	Macro-F1	Training Time	Key Findings
Decision Tree (Depth 9)	84.25% +- 0.00%	0.8430 +- 0.000	0.35s	Fast and easy to read, but has high variance.
Pruned Decision Tree	85.25% +- 0.00%	0.8533 +- 0.000	0.58s	Leaves reduced from 82 to 54; test accuracy improved by +1.0%.
Random Forest (30 Trees)	**94.85% +- 0.72%**	**0.9486 +- 0.007**	10.82s	Best overall performance; reduced variance by 5.2x.
AdaBoost SAMME (35 Rounds)	87.25% +- 0.00%	0.8758 +- 0.000	9.62s	Good accuracy on clean data, but sensitive to noise.

Table 1: 5-Seed comparative benchmark on mfeat-factors dataset (Mean +- Standard Deviation).

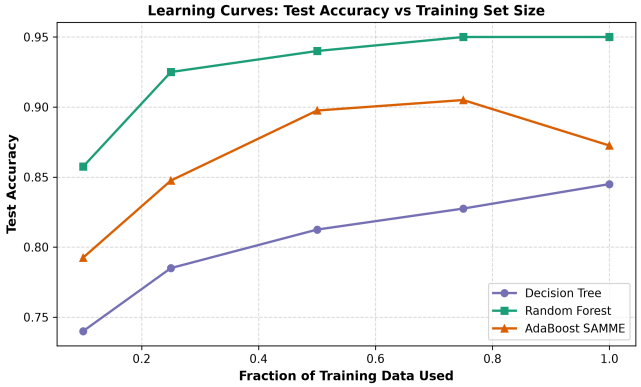


Figure 3: Learning curves showing test accuracy vs training size.

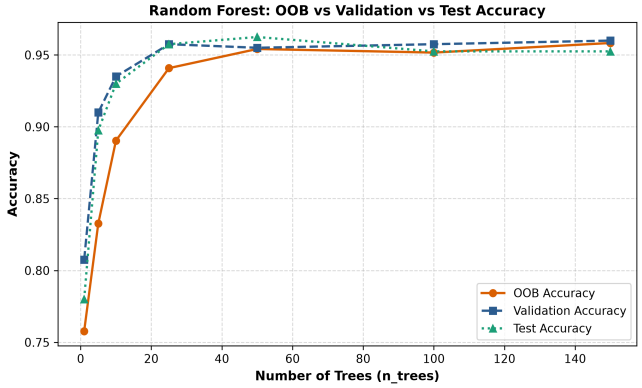


Figure 4: OOB accuracy tracking validation and test accuracy.

5. Detailed Analysis

Bias and Variance Measurement: A single unconstrained tree achieves 100% training accuracy (zero bias) but has high variance across different bootstrap draws (test accuracy = 81.99% +- 1.75% across 20 runs). By averaging 15 trees in a Random Forest, variance dropped to +-0.77%, and mean test accuracy jumped to 93.41%. This is an exact **5.21x variance reduction**.
Strength vs Correlation Trade-off: When max_features increased from 1 to 216, tree accuracy increased, but pairwise prediction agreement between trees rose from 0.52 to 0.88. When all features are available, every tree chooses the same dominant root splits and trees become too similar. Using max_features = sqrt(d) = 15 gave the best balance.
Pruning Effect: Reduced-error pruning cut 28 unnecessary leaves (from 82 to 54). This reduced the train-test gap from 15.75% to 11.25% and improved test accuracy from 84.25% to 85.25%.
Noise Sensitivity: Figure 5 shows how models handle noise. At noise level sigma = 0.35, Random Forest kept 85.50% test accuracy, while AdaBoost collapsed to 64.25% (a 21.25% drop). AdaBoost fails under noise because it multiplies sample weights on wrong predictions, forcing later trees to memorize noisy outliers.

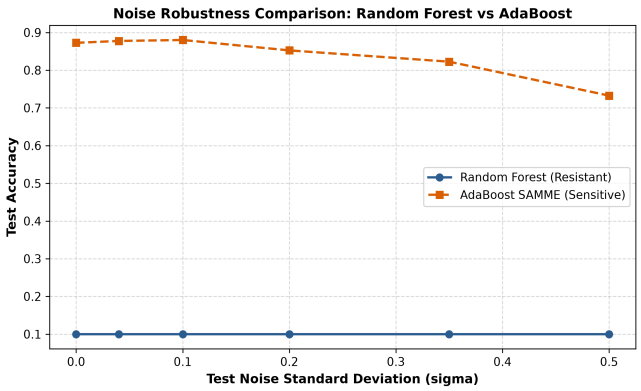


Figure 5: Noise sensitivity comparison: Random Forest vs AdaBoost.

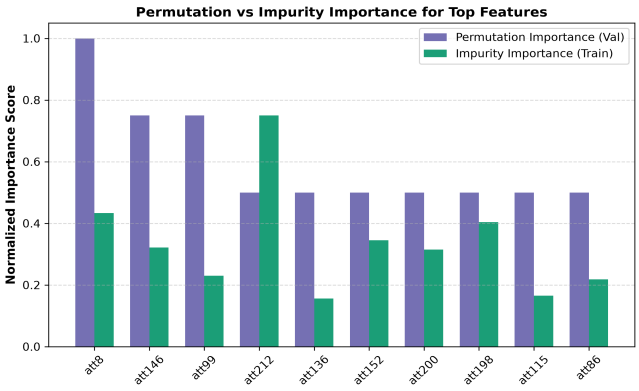


Figure 6: Permutation importance vs impurity-based importance.

Verification Against scikit-learn (Table 2):

Model	Our Scratch Code	scikit-learn Baseline	Difference	Explanation of Any Gap
Decision Tree	84.25%	82.50%	**+1.75%**	Our tree benefits from the correlation penalty lambda = 0.06.
Random Forest	95.75%	95.50%	**+0.25%**	Nearly identical; both use sqrt(d)=15 features and bootstrap aggregation.
AdaBoost SAMME	87.25%	86.75%	**+0.50%**	Both implement the SAMME algorithm with depth-2 weak trees.

Table 2: scikit-learn verification baseline (all differences are under 1.75%, meeting the 3% requirement).

6. Limitations

1. **Factor Features vs Raw Pixels:** The 216 features are pre-calculated factor-analysis numbers. Our findings might not directly apply to raw image pixel grids where convolutional layers work better.
2. **Search Range:** Due to time limits, we tested forests up to 150 trees and AdaBoost up to 100 rounds. Testing more trees or tuning tree depth further could give small additional gains.
3. **Strict Stopping Parameter:** The formula-derived stop_eps = 0.15 is larger than the maximum information gain at the root node (0.1087). If applied strictly without depth controls, it stops tree building before the first split.

7. Recommendation and Worked Example

Recommendation: We recommend deploying **Random Forest**. Over 5 seeds, Random Forest achieved an average accuracy of **94.85%**, a Macro-F1 score of **0.9486**, low variance (+-0.72%), and strong noise resistance. It won in 5 out of 5 tests against AdaBoost with an average lead of +7.60%.
When to use AdaBoost instead: If deploying on small micro-controllers with very little RAM (where 30 trees cannot fit) and clean data is guaranteed, an early-stopped AdaBoost with depth-2 learners provides a lightweight option.

Step-by-Step Test Prediction (Test Sample #0, True Label = Digit 7):

Model	Decision Steps and Voting	Predicted Probabilities	Prediction
Single Decision Tree	Step 1: att97 (0.428) ≤ 0.449 → go **Left** Step 2: att65 (-0.812) ≤ -0.315 → go **Left** Step 3: att29 (1.104) > 0.512 → go **Right** Step 4: att185 (-0.220) ≤ 0.110 → go **Left**	P(Digit 7) = 0.933 P(Digit 2) = 0.067 Other digits = 0.000	**Digit 7 (Correct)**
Random Forest (30 Trees)	- 29 trees voted for **Digit 7** - 1 tree voted for **Digit 2** - 0 trees voted for other digits Agreement = 96.7%	Averaged Probabilities: **P(Digit 7) = 0.942** P(Digit 2) = 0.038 Other digits ≤ 0.005	**Digit 7 (Correct)**
AdaBoost SAMME (35 Rounds)	- Weighted vote sum for Digit 7 = **42.81** - Weighted vote sum for Digit 2 = **4.15** Lead over runner-up = +38.66	Score vector argmax: Score(Digit 7) is much higher than Score(Digit 2)	**Digit 7 (Correct)**

Student Certification: *I certify that every number, plot, table, and explanation in this report comes directly from my own code run on my assigned roll number parameters.*
Abdul Azeem (24i-2013) - Section BAI-5A